MINI PROJECT REPORT ON

"Building of website Using Agile Techniques and pipelining"

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING BY

Gyaan Achantani (T11,01)

Devashish Ahuja (T11,02)

Yogini Bhatia (T11,13)

SUBJECT

Agile Software Development and DevOps

SUPERVISOR

Prof. Sanober shaikh

DEPARTMENT OFARTIFICIAL INTELLIGENCE & DATASCIENCE THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 UNIVERSITY OF MUMBAI

[2025-26]



THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled **"Building of website Using Agile Techniques and pipelining"** is a bonafide work of **Gyaan Achantani, Devashish Ahuja, Yogini Bhatia** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science"**

Prof. Sanober Shaikh

(Supervisor)

TABLE OF CONTENTS

Contents	Page Numbers
1. Aim	 4
2. Mini Project Topic	 4
3. Theory	 5
4. Conclusion	 6

1. AIM

To design and develop a responsive website using Agile software development techniques and implement a CI/CD pipeline to automate the processes of version control, build, testing, and deployment, thereby enabling efficient, iterative, and continuous delivery of the web application.

The aim of the Vital Care project is to design, implement, and deploy an enterprise-grade, full-stack healthcare application integrated with an automated continuous integration and continuous deployment (CI/CD) pipeline. By combining a robust Spring Boot backend, a responsive React frontend, and a centralized MySQL database deployed via Docker Compose, the platform ensures high availability, modular scalability, and secure data management. Furthermore, the architecture incorporates real-time infrastructure monitoring through Grafana and automated build orchestration via Jenkins, seamlessly linked with Jira issue tracking to enforce rigorous software engineering practices. Ultimately, Vital Care seeks to streamline clinical workflows, eliminate manual deployment overhead, and provide reliable telemetry for resilient healthcare operations.

2. MINI – PROJECT TOPIC

Vital Care: Integrated CI/CD and Real-Time Infrastructure Monitoring Pipeline

The **Vital Care** project is a full-stack health application engineered to demonstrate modern DevOps practices, automated containerization, and real-time infrastructure observability. Built around a Spring Boot backend and React frontend, the application is orchestrated using Docker Compose to ensure seamless multi-container deployment across local environments.

The core highlight of this project is its fully integrated continuous integration and deployment (CI/CD) pipeline powered by Jenkins, GitHub, and Jira. Code changes are managed through feature branch workflows linked directly to issue tracking. When a commit is pushed, Jenkins triggers automated build and deployment sequences, ensuring rapid iteration and reliability.

To maintain high availability and performance visibility, the architecture incorporates containerized telemetry tools like Grafana. Connected securely via Docker host networking to a persistent MySQL database backend, Grafana provides live metrics dashboards. This end-to-end integration of automated testing, container deployment, and infrastructure monitoring mirrors industry-standard enterprise software engineering workflows. The system aims to provide healthcare professionals with a centralized platform for accessing and updating patient data, facilitating timely monitoring, improved decision-making, and efficient healthcare management.

3. THE AGILE THEORY

The proposed Patient Healthcare Monitoring System is developed using Agile Software Development and DevOps practices to achieve an efficient, reliable, and continuously deliverable software solution. Agile methodology supports iterative development by dividing the project into manageable tasks and allowing continuous improvement based on changing requirements. Jira is used for Agile project management by organizing requirements and development activities into issues, backlog items, sprints, and workflow stages, thereby providing visibility and coordination among team members. The integration of Jira with the development and DevOps workflow enables project activities and software development progress to be tracked systematically.

GitHub is used as the source code management platform for maintaining the healthcare application's source code and version history. Developers can commit and push changes to the shared repository, enabling collaborative development and maintaining a centralized version-controlled codebase. GitHub can be integrated with Jenkins so that changes made to the repository can initiate automated software development activities. Jenkins is an open-source automation server used for implementing Continuous Integration (CI) and Continuous Delivery/Deployment (CD). It automates important stages such as obtaining the latest source code, building the application, testing it, packaging it, and deploying it. A Jenkins Pipeline represents these activities as a sequence of automated stages and can be defined using a Jenkinsfile, making the development and deployment process repeatable and consistent.

Docker is used to containerize the Patient Healthcare Monitoring System and package the application together with its dependencies, libraries, configuration, and runtime environment into lightweight and isolated containers. Containerization ensures that the application behaves consistently across development, testing, and deployment environments. Docker also improves portability, scalability, and resource utilization.

For a healthcare application consisting of multiple services, Docker Compose is used to define and manage the complete application stack. A Compose configuration can specify application services, databases, networks, ports, environment variables, volumes, and dependencies. It allows multiple containers to be created, started, monitored, stopped, restarted, and removed as a single application environment. This simplifies the deployment and management of the different components of the Patient Healthcare Monitoring System.

Grafana is incorporated as a monitoring and visualization platform for the deployed application and its infrastructure. It can be used to present system and application metrics through interactive dashboards, helping developers and administrators observe service performance, resource utilization, availability, and other operational parameters. Monitoring provides better visibility into the running system and helps identify performance issues or abnormal behavior at an early stage.

The overall implementation follows a DevOps pipeline integrated with Jira, connecting Agile project management with source-code management, automated CI/CD, containerized deployment, and monitoring. A typical workflow begins with requirements and tasks being managed in Jira, followed by development and version control through GitHub. Changes are then processed by Jenkins, where automated Build, Test, and Deployment stages are executed. Docker and Docker Compose provide a consistent containerized deployment environment, while Grafana provides monitoring and visualization of the deployed system. This integration creates a continuous workflow from planning → development → version control → build → testing → deployment → monitoring, improving automation, traceability, collaboration, and reliability throughout the software development lifecycle.

4. LAB OUTCOMES

A. LO-03 - Understand and implement Agile Software Development and DevOps practices for software development and deployment.

B. LO-04 - Understand and implement containerization and multi-service application deployment using Docker and Docker Compose.

C. LO-05 - Implement Continuous Integration and Continuous Delivery /Deployment using automation tools such as Jenkins and GitHub.

5. CONCLUSION

The Vital Care project successfully establishes an integrated, containerized full-stack deployment and monitoring architecture. By combining a Spring Boot backend, React frontend, and Grafana monitoring within a Docker Compose environment, the system ensures seamless scalability and reliable performance.

A core achievement of this implementation is the robust integration of automated CI/CD workflows using Jenkins and issue tracking via Jira, streamlining the software development lifecycle from code commit to deployment. Furthermore, configuring persistent Docker volumes and establishing secure network bindings via host.docker.internal resolved critical database connectivity and authentication challenges with the host MySQL instance.

Ultimately, this project delivers a resilient, production-grade monitoring and deployment pipeline that enhances system observability, automates build tasks, and maintains high availability for health-care application data.

The project demonstrated an automated and efficient workflow from development and testing to deployment and monitoring, improving collaboration, reliability, and maintainability of the healthcare application.